

Parallel Intrusion Detection System (IDS) Log Analyzer
Deteksi Serangan Siber Paralel Menggunakan OpenMP dan OpenCL



Tim Penyusun :

Yoga Candra Putra	(25032014053)
Mutia Alma Sovia	(25032014058)
Muhammad Ibnu Reza	(25032014031)

Dosen Pengampu :

Harmon Prayogi, M. Sc. (0026069206)

KECERDASAN ARTIFISIAL
FAKULTAS
MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2026

Bab I - Pendahuluan

1.1 Latar Belakang

Keamanan jaringan komputer menjadi salah satu tantangan terbesar di era digital. Setiap detik, sistem jaringan modern menghasilkan ribuan hingga jutaan baris log yang merekam aktivitas lalu lintas data. Di dalam aliran log tersebut tersembunyi [1] pola-pola serangan siber seperti *SQL Injection*, *Cross-Site Scripting (XSS)*, *DDoS Flood*, hingga *Command Injection* yang jika tidak terdeteksi tepat waktu dapat menimbulkan kerugian besar [2].

Pendekatan tradisional menganalisis log secara sekuensial satu baris per satu tidak lagi memadai ketika volume data mencapai ratusan ribu hingga jutaan entri per menit [3]. Dibutuhkan sistem yang mampu memparalelkan proses analisis secara efisien memanfaatkan sumber daya komputasi heterogen: *multi-core CPU* dan *GPU* [4].

Proyek ini mengimplementasikan dan membandingkan empat pendekatan analisis log secara paralel menggunakan teknologi *heterogeneous computing*: *Sequential (baseline)*, *OpenMP (parallelisme CPU)*, *OpenCL (parallelisme GPU)*, dan *Hybrid (pipeline OpenMP + OpenCL)* [4]. Pengukuran empiris dilakukan untuk membuktikan manfaat *parallelisme* sekaligus mengidentifikasi *bottleneck* yang membatasi *speedup*, dikaitkan dengan prinsip Hukum Amdah [5].

1.2 Rumusan Masalah

- Seberapa besar peningkatan throughput yang dicapai dengan memparalelkan analisis log menggunakan OpenMP dan OpenCL dibanding pendekatan sekuensial?
- Bagaimana perbedaan arsitektur *CPU-parallelism* (OpenMP) dan *GPU-parallelism* (OpenCL) mempengaruhi performa pada beban kerja *pattern matching*?
- Mengapa pendekatan Hybrid (gabungan OpenMP dan OpenCL) tidak selalu menghasilkan *speedup* tertinggi, dan faktor apa yang menjadi *bottleneck*?

1.3 Tujuan

1. Mengimplementasikan empat varian analisis log: *Sequential*, *OpenMP*, *OpenCL*, dan *Hybrid* dalam satu *codebase* terstruktur.
2. Mengukur dan membandingkan performa keempat varian secara empiris: waktu eksekusi, *throughput*, dan *speedup*.
3. Menganalisis penyebab ketidaklinearan *speedup* menggunakan Hukum Amdahl dan mengidentifikasi *bottleneck* pada *pipeline* Hybrid.
4. Mengevaluasi *trade-off* antara kompleksitas implementasi dan keuntungan performa pada masing-masing pendekatan.

1.4 Batasan Proyek

- Dataset yang digunakan adalah log jaringan sintesis berjumlah 100.000 entri yang dibangkitkan oleh program `log_generator.c`.
- Pengujian dilakukan pada platform Linux dengan CPU 2-core; OpenCL menggunakan POCL (*Portable Computing Language*) sebagai CPU *fallback* karena tidak tersedia GPU fisik.
- Fokus utama adalah *pattern matching* berbasis *signature* (15 pola); deteksi anomali statistik berada di luar cakupan proyek ini.

Bab II - Dasar Teori

2.1 Heterogeneous Computing

Heterogeneous computing adalah paradigma komputasi yang menggunakan lebih dari satu jenis unit pemrosesan dalam satu sistem umumnya kombinasi CPU (Central Processing Unit) dan GPU (*Graphics Processing Unit*) atau akselerator lain [4]. Setiap unit memiliki karakteristik berbeda:

Karakteristik	CPU	GPU
Jumlah Core	Sedikit (2–64 core), kompleks	Ribuan (ratusan–ribuan), sederhana
Optimasi	Latensi rendah, kontrol flow kompleks	<i>Throughput</i> tinggi, beban seragam
Cocok untuk	Task bercabang, serial, I/O intensif	<i>Data-parallel</i> : array, matrix, gambar
Model memori	Cache hierarki (L1/L2/L3), shared	Global/local/constant/private memory

2.2 OpenMP (Open Multi-Processing)

OpenMP adalah antarmuka pemrograman paralel berbasis *shared-memory* untuk CPU *multi-core*. *Paralelisme* dinyatakan melalui *pragma* (*directive*) langsung di kode C/C++/Fortran tanpa mengubah struktur program secara fundamental.

Konsep-konsep kunci OpenMP yang digunakan dalam proyek ini:

- `parallel for`: membagi iterasi loop ke beberapa thread secara otomatis.
- `schedule(dynamic, chunk)`: strategi pembagian kerja dinamis setiap *thread* mengambil *chunk* iterasi dari antrian saat selesai; cocok untuk beban tidak merata [6].
- `reduction(+;var)`: setiap thread akumulasi variabel lokal; digabung di akhir tanpa race condition
- `critical section`: blok kode yang hanya boleh dieksekusi satu thread pada satu waktu.
- `Thread-local buffer`: setiap thread memiliki array histogram sendiri untuk menghindari false sharing pada cache [6].

2.3 OpenCL (Open Computing Language)

OpenCL adalah standar terbuka untuk pemrograman pada hardware heterogen (GPU, CPU, FPGA)[7]. Program OpenCL terdiri dari host code (C/C++ di CPU) dan kernel code (OpenCL C di device).

Hierarki eksekusi OpenCL:

- `NDRange`: ruang indeks N-dimensi yang mendefinisikan total *work-item*.
- *Work-item*: unit eksekusi terkecil; analog dengan thread GPU. Setiap *work-item* mendapat ID unik via `get_global_id(0)`.
- *Work-group*: kumpulan *work-item* yang dapat berbagi local memory dan sinkronisasi; ukuran optimal = kelipatan *warp/wavefront*.

Hierarki memori OpenCL yang digunakan:

- `__global memory`: memori utama GPU, aksesibel semua *work-item*; *latency* tinggi, *bandwidth* besar.
- `__constant memory`: *read-only*, di-cache hardware, di-broadcast ke semua *work-item* dalam satu *work-group* secara gratis [8].

- `__private memory: register per work-item`; tercepat namun terbatas

2.4 Hukum Amdahl

Hukum Amdahl menyatakan bahwa speedup maksimum sebuah program yang diparalelkan dibatasi oleh porsi yang tidak dapat diparalelkan [5]. Secara matematis:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

Di mana: $S(n)$ = *speedup* dengan n *prosesor*

p = fraksi kode yang dapat diparalelkan

$(1-p)$ = bagian sekuensial

Implikasi praktis: jika hanya 80% kode yang dapat diparalelkan, maka dengan 2 *prosesor speedup* maksimum hanya $1/(0.2 + 0.8/2) = 1.67x$ bukan $2x$. Dengan 8 prosesor pun hanya $3.08x$. Ini menegaskan bahwa mengidentifikasi dan mengoptimasi bagian sekuensial jauh lebih kritis dari sekadar menambah prosesor [5].

2.5 Algoritma Boyer-Moore Horspool

Boyer-Moore Horspool adalah varian disederhanakan dari algoritma *Boyer-Moore* untuk pencarian substring [6]. Kelebihan utamanya adalah kemampuan melewati (*skip*) karakter yang tidak cocok menggunakan *lookup table*, sehingga rata-rata kompleksitas waktu mencapai $O(n/m)$ untuk teks acak.

Dalam proyek ini, algoritma ini digunakan di versi *Sequential* dan OpenMP karena efisien pada CPU. Versi OpenCL menggunakan *naive substring search* yang lebih sederhana namun lebih ramah terhadap divergensi kontrol di GPU [9].

Bab III - Desain Dan Implementasi

3.1 Arsitektur Sistem

Sistem terdiri dari tiga lapisan utama yang bekerja secara berurutan:

Lapisan	Deskripsi
Log Generator	Membangkitkan dataset sintetis 100.000 entri log jaringan. Setiap entri berisi: <i>timestamp</i> , IP sumber/tujuan, <i>port</i> , protokol, <i>bytes</i> , dan <i>payload</i> . Sekitar 15% entri mengandung pola serangan yang disisipkan secara acak.
Log Loader	Parser file log → <i>array struct LogEntry</i> . Setiap baris teks diparsing menjadi 8 <i>field</i> terstruktur yang siap diproses oleh <i>engine</i> .
Engine Analisis	Empat implementasi <i>engine</i> analisis (<i>Sequential</i> , OpenMP, OpenCL, Hybrid) yang melakukan <i>pattern matching</i> terhadap 15 <i>signature</i> serangan. <i>Output</i> : daftar <i>threat</i> , histogram <i>severity</i> , dan data <i>benchmark</i> performa.

3.2 Struktur Data

File `ids_common.h` mendefinisikan tiga struktur data inti:

```
typedef struct {
    char timestamp[32];    // Format: YYYY-MM-DD HH:MM:SS
    char src_ip[16];       // IP sumber serangan
    char dst_ip[16];       // IP tujuan
    int src_port, dst_port;
    char protocol[8];      // TCP/UDP/ICMP
```

```

    int  bytes;           // Ukuran paket
    char payload[256];    // Data yang dianalisis fixed 256B untuk GPU
} LogEntry;

```

3.3 Signature Serangan

Sistem menggunakan 15 *signature* serangan yang didefinisikan secara statis di `ids_common.h`, dibagi dalam empat tingkat keparahan:

No.	Nama Serangan	Signature	Severity
1	SQL Injection	' OR '1'='1	Critical (4)
2	SQL Injection UNION	UNION SELECT	Critical (4)
3	XSS Attack	<script>	High (3)
4	XSS Alert	alert(	High (3)
5	Path Traversal	../../../../	High (3)
6	Command Injection	;rm -rf	Critical (4)
7	Command Injection 2	nc	Critical (4)
8	Brute Force SSH	Failed password	Medium (2)
9	Port Scan	SYN_SCAN	Medium (2)
10	DDoS Flood	FLOOD	Critical (4)
11	Malware C2	/cmd.php	Critical (4)
12	Backdoor	/shell.php	Critical (4)
13	LFI Attack	/etc/passwd	High (3)
14	RFI Attack	http://evil	High (3)
15	Buffer Overflow	AAAAAAAAAAAAAAAA	Critical (4)

3.4 Implementasi Varian Analisis

3.4.1 Sequential (Baseline)

Implementasi sekuensial memproses *log* satu per satu dalam *loop* tunggal menggunakan algoritma *Boyer-Moore Horspool*. Setiap *log* dicocokkan terhadap 15 pola secara berurutan. Tidak ada *paralelisme*.

Kompleksitas: $O(n \times m \times P)$ di mana n = jumlah *log*, m = panjang *payload* rata-rata, P = jumlah *pattern* (15).

3.4.2 OpenMP (CPU Parallel)

Versi OpenMP memparalelkan *loop* utama analisis *log* menggunakan *directive*:

```

#pragma omp parallel
{
    int local_severity[5] = {0};    // buffer thread-local

```

```

int local_pattern[NUM_PATTERNS] = {0};

#pragma omp for schedule(dynamic, 256) reduction(+:threat_count)
for (int i = 0; i < n; i++) {
    for (int p = 0; p < NUM_PATTERNS; p++) {
        if (contains_pattern(logs[i].payload, PATTERNS[p].signature))
            threat_count++; local_severity[...]++;
    }
}
#pragma omp critical { /* merge histogram lokal ke global */ }
}

```

Keputusan desain yang diambil:

- *schedule(dynamic, 256)*: *chunk* 256 log per ambil *balancing* antara *overhead scheduling* dan *granularitas*. *Chunk* terlalu kecil → *overhead* tinggi; terlalu besar → *imbalance*.
- *Buffer thread-local* (*local_severity*, *local_pattern*): menghindari *false sharing* pada histogram global. Setiap *thread* update *array*-nya sendiri, *merge* sekali di akhir.
- *reduction(+:threat_count)*: *compiler* menangani akumulasi *counter* secara aman tanpa *critical section*.
- *critical* hanya untuk menulis ke *array threats[]* yang butuh posisi konsisten.

3.4.3 OpenCL (GPU Parallel)

Versi OpenCL memindahkan beban *pattern matching* ke GPU. Kernel *detect_attacks* di *ids_kernel.cl* mengimplementasikan paradigma "satu work-item = satu log":

```

__kernel void detect_attacks(
    __global const char *payloads,          // input: semua payload
    __constant const char *patterns,        // pattern di constant memory (cached)
    __constant const int *severities,
    const int num_logs, const int num_patterns,
    __global uchar *match_matrix,          // output: [n x P] biner
    __global int *threat_per_log,          // jumlah hit per log
    __global int *severity_per_log)        // severity max per log
{
    int gid = get_global_id(0);            // ID unik tiap work-item
    if (gid >= num_logs) return;
    // setiap work-item proses payload-nya sendiri vs semua pattern
}

```

Keputusan desain kunci:

- Stride tetap 256 *byte/log* untuk *payload*: memastikan *coalesced memory access work-item* berurutan membaca alamat memori berurutan, memaksimalkan *bandwidth* GPU.
- *__constant* memory untuk *patterns* dan *severities*: GPU men-*cache* data ini dan mem-*broadcast* ke seluruh *work-item* dalam satu *work-group* tanpa biaya ekstra.
- *Work-group* size 64: kelipatan *warp* (32) NVIDIA atau *wavefront* (64) AMD memaksimalkan *occupancy* GPU.
- *Output match_matrix* biner [n×P]: agregasi dilakukan di *host* (CPU) setelah transfer balik, bukan di kernel.

3.4.4 Hybrid Pipeline (OpenMP + OpenCL)

Versi Hybrid menggabungkan keduanya dalam *pipeline* tiga tahap yang berurutan:

Tahap	Teknologi & Fungsi	Waktu (100K log)
1. Preprocess	OpenMP: normalisasi <i>payload</i> (<i>lowercase</i> , strip <i>whitespace</i>), <i>flatten</i> ke <i>stride</i> tetap 256B	0.019s (2,4%)
2. Matching	OpenCL: kernel GPU cocokkan semua <i>log</i> vs semua <i>pattern</i> secara masif paralel	0.052s (6,5%)
3. Agregasi	OpenMP: baca <i>match_matrix</i> hasil GPU, bangun histogram, ranking IP penyerang	0.728s (91,1%) - Bottleneck

Bab IV - Hasil Pengujian Dan Analisis

4.1 Lingkungan Pengujian

Komponen	Spesifikasi
Sistem Operasi	Linux (Ubuntu 24.04)
Kompiler	GCC dengan flag <code>-fopenmp -O2</code>
OpenMP	Versi 4.5, 2 thread aktif
OpenCL	OpenCL 1.2 via POCL (CPU fallback, tidak ada GPU fisik)
Dataset	100.000 entri log jaringan sintetis (~15% mengandung serangan)
Pengulangan	Setiap varian dijalankan 3 kali; nilai yang dilaporkan adalah rata-rata
Pengukuran waktu	<code>clock_gettime(CLOCK_MONOTONIC)</code> resolusi nanosecond

4.2 Hasil Benchmark

Tabel berikut merangkum hasil pengujian keempat varian pada dataset 100.000 log:

Varian	Waktu (detik)	Throughput (log/s)	Speedup vs Seq	Threat Ditemukan
Sequential	0.2377	420.690	1.00x (baseline)	17.481
OpenMP (2T)	0.1149	870.126	2.07x	17.481
OpenCL (kernel)	0.1088	1.226.759	2.19x	17.481
OpenCL (total GPU)	0.1308 (incl. H↔D)	764.526	1.82x	17.481
Hybrid (total)	0.7992	125.132	0.30x - lebih lambat	17.481

Semua varian menghasilkan jumlah threat yang identik (17.481 dari 100.000 log = 17,48%) - membuktikan kebenaran (*correctness*) semua implementasi paralel.

4.3 Analisis Speedup dan Efisiensi

Efisiensi paralel didefinisikan sebagai *speedup* dibagi jumlah unit prosesor yang digunakan:

Varian	Speedup	Prosesor	Efisiensi
OpenMP 2 thread	2.07x	2 CPU core	103.5% *
OpenCL kernel	2.19x	~16 work-group	13.7% per WG
Hybrid total	0.30x	CPU + GPU	< 0 (kerugian)

Efisiensi OpenMP > 100% dimungkinkan karena: (1) POCL menggunakan lebih dari 2 *thread virtual* secara internal, (2) cache warm-up effect pada run sequential, (3) *schedule(dynamic)* memberikan *load balancing* yang lebih baik dari ekspektasi [10].

4.4 Analisis Bottleneck Hybrid Hukum Amdahl

Hasil paradoks *Hybrid* (lebih lambat dari sequential) dapat dijelaskan melalui dekomposisi *pipeline*:

Tahap	Waktu	% Total	Bisa Diparalelkan?
Preprocess (OMP)	0.019s	2.4%	Ya (sudah)
Kernel (OpenCL)	0.052s	6.5%	Ya (sudah)
Agregasi (OMP)	0.728s	91.1%	Terbatas (omp critical)

Akar penyebab *bottleneck* Tahap 3:

- Setiap *thread* harus membaca seluruh *match_matrix* hasil GPU ($100.000 \times 15 = 1,5$ juta byte) dari memori *bandwidth bottleneck* [4].
- Untuk setiap log yang mengandung *threat*, *thread* harus masuk ke *#pragma omp critical* untuk menulis ke *array* global dan update hash table IP - serialisasi paksa [6].
- Dengan 17.481 threat dari 100K log, berarti sekitar 17.481 kali masuk critical section - overhead sinkronisasi yang signifikan.

Proyeksi Hukum Amdahl dengan $p = 0.089$ [5] (hanya 8.9% kode yang berjalan paralel efektif):

$$S(2) = \frac{1}{(1 - 0.089) + \frac{0.089}{2}} = \frac{1}{0.911 + 0.0445} = \frac{1}{0.9555} = 1.047x$$

Ini mendekati hasil aktual 0.30x (lebih rendah karena overhead transfer H↔D dan pipeline synchronization melebihi keuntungan paralelisme).

4.5 Perbandingan Memori dan Transfer Cost

Operasi	Ukuran Data	Waktu
Host → Device (payload)	$100K \times 256B = 25.6$ MB	0.032s
Kernel execution	100K work-items	0.052s
Device → Host (matrix)	$100K \times 15 \times 1B = 1.5$ MB	~0.008s

Total OpenCL pipeline	-	~0.092s
-----------------------	---	---------

Transfer H→D memakan 0.032 detik lebih lama dari eksekusi kernel itu sendiri (0.052 detik) . Ini konsisten dengan prinsip bahwa GPU hanya menguntungkan jika komputasi yang dilakukan cukup besar untuk mengamortisasi biaya transfer [11]. Untuk *dataset* 100K log, rasio sudah di atas ambang batas, tapi untuk *dataset* lebih kecil (<10K log) *overhead* transfer akan mendominasi.

Bab V - Evaluasi Sistem

5.1 Kelebihan Sistem

Kelebihan sistem pada proyek kami adalah:

Aspek	Kelebihan
Correctness	Keempat varian menghasilkan output identik (17.481 threat), membuktikan semua implementasi paralel bebas <i>race condition</i> dan memberikan hasil yang benar
Performa	OpenMP 2.07x dan OpenCL kernel 2.19x <i>speedup</i> terhadap <i>baseline</i> , peningkatan throughput nyata tanpa mengorbankan akurasi
Memory optimization	Stride-tetap 256B untuk <i>coalesced</i> GPU access; <code>__constant</code> memory untuk pattern; <i>buffer thread-local</i> untuk histogram OpenMP, ketiganya berkontribusi pada performa
Portabilitas	OpenCL berjalan di CPU, GPU, FPGA. POCL <i>fallback</i> memungkinkan pengujian tanpa <i>hardware</i> GPU, penting untuk <i>environment development</i> .
Modularitas	Kode terstruktur: <code>ids_common.h</code> sebagai <i>foundation</i> bersama; setiap varian dalam file terpisah; Makefile dengan target <i>benchmark</i> otomatis

5.2 Keterbatasan dan Kendala

Keterbatasan dan kendala yang kami temui selama pengerjaan proyek ini:

Keterbatasan	Dampak dan Penjelasan
Tidak ada GPU fisik	POCL berjalan di CPU, sehingga hasil "GPU" sebenarnya adalah CPU juga. Akibatnya, <i>speedup</i> OpenCL tidak mencerminkan potensi penuh GPU nyata yang memiliki ribuan core. GPU fisik berpotensi memberikan <i>speedup</i> 10-50x untuk beban kerja serupa
Bottleneck agregasi	Tahap agregasi <i>Hybrid</i> belum dioptimasi: masih menggunakan <code>omp critical</code> untuk setiap penulisan threat. Ini menciptakan titik serialisasi yang mendominasi 91% waktu pipeline
Algoritma matching naive di GPU	Kernel menggunakan <i>naive substring search</i> $O(n \times m)$, efisien untuk 15 pattern pendek, tapi tidak skalabel jika jumlah <i>pattern</i> bertambah besar
Dataset sintetis	<i>Log</i> dibangkitkan secara acak dengan distribusi tetap, distribusi <i>payload</i> dunia nyata lebih variatif dan mungkin mempengaruhi efektivitas <i>schedule(dynamic)</i>

5.3 Rencana Optimasi Lanjutan

Berdasarkan analisis *bottleneck*, berikut adalah prioritas optimasi yang direkomendasikan:

Prioritas	Optimasi
1. Kritis	Ganti <i>omp critical</i> pada agregasi <i>Hybrid</i> dengan <i>atomic operations</i> + <i>reduce</i> bertingkat (<i>tree reduction</i>). Estimasi: potong waktu agregasi dari 0.728s menjadi <0.1s → Hybrid bisa unggul <i>sequential</i>
2. Tinggi	Implementasi algoritma <i>Aho-Corasick</i> di kernel GPU: membangun <i>finite automaton</i> untuk semua <i>P pattern</i> sekaligus, sehingga kompleksitas pencarian turun drastis menjadi $O(n)$ dibandingkan pendekatan <i>naive</i> $O(n \times m \times P)$
3. Sedang	<i>Pinned memory</i> (<i>CL_MEM_ALLOC_HOST_PTR</i>) untuk transfer asinkron $H \rightarrow D$ yang <i>overlap</i> dengan <i>preprocessing</i> OpenMP, menghilangkan <i>dead time</i> antara tahap 1 dan 2
4. Tambahan	<i>Sharding hybrid</i> : bagi <i>dataset</i> 50/50 antara GPU dan CPU OpenMP yang berjalan secara simultan, memanfaatkan kedua resource sepenuhnya

Bab VI - Kesimpulan Dan Saran

6.1 Kesimpulan

Proyek Parallel IDS Log Analyzer berhasil mengimplementasikan dan membandingkan empat varian analisis log jaringan untuk deteksi serangan siber. Beberapa kesimpulan utama:

1. Paralelisme CPU (OpenMP) memberikan *speedup* nyata 2.07x dengan 2 *thread* aktif pada beban kerja *pattern matching*. Penggunaan *schedule(dynamic, 256)* dan *buffer thread-local* terbukti efektif meminimalkan *overhead* sinkronisasi.
2. Paralelisme GPU (OpenCL) menghasilkan *throughput* tertinggi untuk fase kernel: 1.226.759 log/detik (*speedup* 2.19x). Strategi *coalesced memory access* dan *constant memory* untuk *pattern* berkontribusi signifikan.
3. Pendekatan Hybrid tidak selalu superior: *pipeline* OpenMP+OpenCL justru 3.3x lebih lambat dari *sequential* karena *bottleneck* agregasi (91% waktu total) yang belum dioptimasi. Ini adalah demonstrasi nyata Hukum Amdahl bagian sekuensial mendominasi keseluruhan performa.
4. *Speedup* tidak linier disebabkan oleh: (a) *overhead omp critical* dan *false sharing*, (b) biaya transfer $H \leftrightarrow D$ yang melebihi keuntungan kernel, (c) bagian kode yang inheren sekuensial seperti I/O dan agregasi global.
5. Hal yang kami pelajari: identifikasi *bottleneck* sebelum memparalelkan. Mengoptimasi tahap yang hanya mengambil 9% waktu (*preprocess* + kernel) tidak mengubah gambaran besar; fokus harus pada agregasi yang mengambil 91%.

6.2 Saran

- Untuk produksi: implementasikan *Aho-Corasick* di kernel GPU dan *tree reduction* di agregasi OpenMP sebelum *deployment* di lingkungan dengan volume log tinggi.
- Untuk penelitian lanjutan: uji pada GPU fisik (misalnya NVIDIA RTX atau AMD RX series) untuk mengukur potensi *speedup* sesungguhnya yang tidak terbatas oleh POCL.
- Untuk skalabilitas: tambahkan modul deteksi anomali statistik (*z-score traffic* per IP) sebagai komplemen *signature-based* serangan *zero-day* tidak memiliki *signature*.

- Untuk ketahanan: implementasikan *hot-reload signature* tanpa *restart* sistem untuk merespons *threat intelligence* terbaru secara *real-time*.

Daftar Referensi

- [1] P. Notaro, S. Haeri, J. Cardoso, and M. Gerndt, "LogRule: Efficient Structured Log Mining for Root Cause Analysis," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 4231–4243, Dec. 2023, doi: 10.1109/TNSM.2023.3282270.
- [2] M. Çelebi, A. Özbilen, and U. Yavanoğlu, "A comprehensive survey on deep packet inspection for advanced network traffic analysis: issues and challenges," *Nigde Ömer Halisdemir University Journal of Engineering Sciences*, vol. 12, no. 1, pp. 1–29, Jan. 2023, doi: 10.28948/NGUMUH.1184020.
- [3] A. Lazar, A. Sim, and K. Wu, "GPU-based Classification for Wireless Intrusion Detection," *SNTA 2021 - Proceedings of the 2021 Systems and Network Telemetry and Analytics, co-located with HPDC 2021*, pp. 27–31, Jun. 2021, doi: 10.1145/3452411.3464445;CTYPE:STRING:BOOK.
- [4] M. Alaei and F. Yazdanpanah, "A Survey on Heterogeneous CPU–GPU Architectures and Simulators," *Concurr. Comput.*, vol. 37, no. 1, p. e8318, Jan. 2025, doi: 10.1002/CPE.8318;WGROU:STRING:PUBLICATION.
- [5] M. A. N. Al-Hayanni, F. Xia, A. Rafiev, A. Romanovsky, R. Shafik, and A. Yakovlev, "Amdahl's law in the context of heterogeneous many-core systems - A survey," *IET Comput. Digit. Tech.*, vol. 14, no. 4, pp. 133–148, Jul. 2020, doi: 10.1049/IET-CDT.2018.5220.
- [6] A. A. AbdulRazaq, Q. S. Hamad, and A. M. Taha, "Parallel implementation of maximum-shift algorithm using OpenMp," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 22, no. 3, pp. 1529–1539, Jun. 2021, doi: 10.11591/IJEECS.V22.I3.PP1529-1539.
- [7] T. Leppänen, A. Lotvonen, and P. Jämskeläinen, "Cross-vendor programming abstraction for diverse heterogeneous platforms," *Front. Comput. Sci.*, vol. 4, p. 945652, Oct. 2022, doi: 10.3389/FCOMP.2022.945652/TEXT.
- [8] Y. Song, C. Li, S. Xiao, Q. Zhou, and H. Xiao, "A parallel Canny edge detection algorithm based on OpenCL acceleration," *PLoS One*, vol. 19, no. 1, p. e0292345, Jan. 2024, doi: 10.1371/JOURNAL.PONE.0292345.
- [9] M. Çelebi and U. Yavanoğlu, "Accelerating Pattern Matching Using a Novel Multi-Pattern-Matching Algorithm on GPU," *Applied Sciences 2023, Vol. 13, Page 8104*, vol. 13, no. 14, p. 8104, Jul. 2023, doi: 10.3390/APP13148104.
- [10] T. Baumann, M. Noack, and T. Steinke, "Performance Evaluation and Improvements of the PoCL Open-Source OpenCL Implementation on Intel CPUs," *ACM International Conference Proceeding Series*, vol. 12, Apr. 2021, doi: 10.1145/3456669.3456698;PAGE:STRING:ARTICLE/CHAPTER.
- [11] J. Solanti, M. Babej, J. Ikkala, V. K. K. Malamal Vadakital, and P. Jämskeläinen, "PoCL-R: A Scalable Low Latency Distributed OpenCL Runtime," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13227 LNCS, pp. 78–94, 2022, doi: 10.1007/978-3-031-04580-6_6/SAVE-RESEARCH.